Name: Shravani Kurkute **PRN:** 22311926
Roll No.: 421129 **Div:** Btech-A
Subject: NLP

Assignment No. 4
PART-A

Code:

```
import json import os
import re import html
import joblib
import numpy as np import pandas as
pd import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt import seaborn as
sns
from nltk.stem import PorterStemmer

from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS, TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import ComplementNB
from sklearn.svm import LinearSVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score,
GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             confusion_matrix, classification_report, roc_auc_score,
                             matthews_corrcoef)

SEED = 42
np.random.seed(SEED)

# ---------------------------------------- 1. LOAD
DATA_PATH = "data/7817_1.csv"
os.makedirs("results",
exist_ok=True) raw_df =
pd.read_csv(DATA_PATH)
print("=" * 70, "\n1. DATASET INSPECTION\n" + "=" * 70)
```

```
print("Raw shape           :", raw_df.shape)
print("Products (unique)   :", raw_df["name"].nunique())
print("Rating counts       :\n",
raw_df["reviews.rating"].value_counts(dropna=False).sort_index().to_string())
print("Missing review text :", int(raw_df["reviews.text"].isna().sum()))
print("Missing rating      :", int(raw_df["reviews.rating"].isna().sum()))
print("Duplicate review text:",
int(raw_df.duplicated(subset=["reviews.text"]).sum()))

# Build the working frame: review = title + text ; label from the star rating
df = raw_df[["reviews.title", "reviews.text", "reviews.rating", "name"]].copy()
n_raw = len(df)
df = df.dropna(subset=["reviews.text", "reviews.rating"])           # need text AND a rating
n_missing = n_raw - len(df)
df = df[df["reviews.rating"] != 3]                                  # drop neutral 3-star reviews
n_neutral = (n_raw - n_missing) - len(df)
n_before = len(df)
df = df.drop_duplicates(subset=["reviews.text"]).reset_index(drop=True) # same review scraped
for several listings
n_dups = n_before - len(df)
df["review"] = df["reviews.title"].fillna("").astype(str) + " . " + df["reviews.text"].astype(str)
df["sentiment"] = np.where(df["reviews.rating"] >= 4, "positive", "negative") # 4-5 stars =
positive, 1-2 = negative
print(f"Removed {n_missing} rows with missing text/rating, {n_neutral} neutral (3-star) rows, "
     f"{n_dups} duplicate reviews -> {len(df)} records remain")
print("Class distribution :\n", df["sentiment"].value_counts().to_string()) print(f"Imbalance:
{df['sentiment'].value_counts(normalize=True)['positive']:.1%} positive")

# ---------------------------------------- 2. PREPROCESS
STEMMER = PorterStemmer()
NEGATIONS = {"not", "no", "never", "nor", "cannot", "cant", "wont", "dont", "doesnt", "didnt",
             "isnt", "wasnt", "arent", "werent", "wouldnt", "couldnt", "shouldnt", "hasnt",
             "havent", "hadnt", "neither", "nothing", "nobody", "none", "hardly", "barely"}
# keep negations + a few polarity-bearing function words; drop the other stop words
STOP = set(ENGLISH_STOP_WORDS) - NEGATIONS - {"but", "however", "against", "very",
"too"}
EMO_MAP = {":)": " emopos ", ":-)": " emopos ", ":d": " emopos ", "<3": " emopos ",
           "👍": " emopos ", "😊": " emopos ",
           ":(": " emoneg ", ":-(": " emoneg ", ":/": " emoneg ", "👎": " emoneg ",
           "😡": " emoneg ", "😞": " emoneg "}
```

```
URL_RE = re.compile(r"(https?://\S+|www\.\S+)")
TAG_RE = re.compile(r"<[^>]+>")
MENTION_RE = re.compile(r"@\w+")
HASH_RE = re.compile(r"#(\w+)")
ELONG_RE = re.compile(r"(.)\1{2,}")                # coooool -> cool
TOKEN_RE = re.compile(r"[a-z]+|[.!?;,]")           # words + clause-boundary punctuation


def clean_text(text: str) -> str:
    text = html.unescape(str(text))
    text = text.lower()
    for k, v in EMO_MAP.items():                   # emoticons carry sentiment -> keep as tokens
        text = text.replace(k, v)
    text = URL_RE.sub(" ", text)
    text = TAG_RE.sub(" ", text)
    text = MENTION_RE.sub(" ", text)
    text = HASH_RE.sub(r" \1 ", text)              # keep hashtag word, drop
    '#' text = re.sub(r"n't\b", " not", text)      # isn't -> is not
    text = re.sub(r"'(s|re|ve|ll|d|m)\b", "", text)
    text = re.sub(r"['’]", "", text)
    text = ELONG_RE.sub(r"\1", text)
    return text


def preprocess(text: str) -> str:
    """clean -> tokenise -> negation scope marking -> stop-words -> stemming"""
    tokens = TOKEN_RE.findall(clean_text(text))
    out, negated = [], False
    for tok in tokens:
        if tok in ".!?;,":
            negated = False                        # negation scope ends at punctuation
            continue
        if tok in NEGATIONS:
            negated = True
            out.append("not")
            continue
        if tok in STOP or len(tok) < 2:
            continue
        stem = STEMMER.stem(tok)
        out.append(("neg_" + stem) if negated else stem)  # Pang et al. (2002) NOT_ trick
```

```
    return " ".join(out)


df["clean"] = df["review"].apply(preprocess)
print("\n" + "=" * 70, "\n2. PREPROCESSING EXAMPLES\n" + "=" * 70)
for i in range(4):
    print(f"RAW : {df['review'][i][:300]}\nCLEAN : {df['clean'][i][:300]}\n")
df["label"] = (df["sentiment"] == "positive").astype(int)
df = df[df["clean"].str.len() > 0].reset_index(drop=True)          # drop reviews that became empty
df["n_tokens"] = df["clean"].str.split().str.len()
print("Avg tokens/review after cleaning:", round(df["n_tokens"].mean(), 2))

# ---------------------------------------------- 3. SPLIT
X_train, X_test, y_train, y_test, raw_train, raw_test = train_test_split(
    df["clean"], df["label"], df["review"], test_size=0.20, stratify=df["label"],
random_state=SEED)
print(f"\nTrain = {len(X_train)} Test = {len(X_test)}")

# ------------------------------------- 4. FEATURE + MODEL
def tfidf():
    return TfidfVectorizer(ngram_range=(1, 2), min_df=2, max_df=0.95,
                           max_features=20000, sublinear_tf=True)

candidates = {
    "Naive Bayes":         ComplementNB(alpha=0.5),
    "Logistic Regression": LogisticRegression(C=1.0, max_iter=1000, class_weight="balanced",
random_state=SEED),
    "Linear SVM":          LinearSVC(C=1.0, class_weight="balanced",
    random_state=SEED), "Random Forest":
    RandomForestClassifier(n_estimators=200,
class_weight="balanced_subsample", random_state=SEED, n_jobs=-1),
}
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
print("\n" + "=" * 70, "\n3. MODEL COMPARISON (5-fold CV on training set, TF-IDF 1-2
grams)\n" + "=" * 70)
cv_rows = []
for name, clf in candidates.items():
    pipe = Pipeline([("tfidf", tfidf()), ("clf", clf)])
    sc = cross_val_score(pipe, X_train, y_train, cv=skf, scoring="f1_macro")
    cv_rows.append((name, sc.mean(), sc.std()))
    print(f"{name:22s} macro-F1 = {sc.mean():.4f} +/- {sc.std():.4f}")
```

```
cv_df = pd.DataFrame(cv_rows, columns=["model", "cv_f1_mean", "cv_f1_std"])

#                                                -- 5. TUNE LOGISTIC REGRESSION
print("\nGrid-search tuning Logistic Regression    ")
pipe = Pipeline([("tfidf", tfidf()), ("clf", LogisticRegression(max_iter=1000,
class_weight="balanced", random_state=SEED))])
grid = GridSearchCV(pipe,
                    {"clf__C": [0.5, 1, 5, 10, 50],
                     "tfidf__ngram_range": [(1, 1), (1, 2)]},
                    cv=skf, scoring="f1_macro", n_jobs=-1)
grid.fit(X_train, y_train)
print("Best params :", grid.best_params_, " CV macro-F1 =", round(grid.best_score_, 4))
best = grid.best_estimator_

# ---------------------------------------- 6. EVALUATE
y_pred = best.predict(X_test)
y_prob = best.predict_proba(X_test)[:, 1]
metrics = {
    "accuracy": accuracy_score(y_test, y_pred),
    "precision": precision_score(y_test, y_pred),                    # positive class
    "recall":    recall_score(y_test, y_pred),
    "f1":        f1_score(y_test, y_pred),
    "macro_precision": precision_score(y_test, y_pred, average="macro"),
    "macro_recall":    recall_score(y_test, y_pred, average="macro"),
    "neg_precision": precision_score(y_test, y_pred, pos_label=0),     # minority (negative) class
    "neg_recall":    recall_score(y_test, y_pred, pos_label=0),
    "neg_f1":        f1_score(y_test, y_pred, pos_label=0),
    "roc_auc": roc_auc_score(y_test, y_prob),
    "mcc":       matthews_corrcoef(y_test, y_pred),
    "macro_f1": f1_score(y_test, y_pred, average="macro"),
}
cm = confusion_matrix(y_test, y_pred)
print("\n" + "=" * 70, "\n4. TEST-SET EVALUATION\n" + "=" * 70)
for k, v in metrics.items():
    print(f"{k:10s}: {v:.4f}")
print("\nConfusion matrix [rows=true, cols=pred] (neg, pos):\n", cm)
report = classification_report(y_test, y_pred, target_names=["negative", "positive"], digits=4)
print("\n", report)

#       more stable estimate: the test set has very few negatives, so also run
```

```
#  repeated stratified 5-fold CV on ALL cleaned reviews with the tuned model
from sklearn.model_selection import RepeatedStratifiedKFold, cross_val_predict
from sklearn.base import clone
rskf_scores = {"accuracy": [], "macro_f1": [], "neg_recall": [], "neg_precision": [], "neg_f1":
[]} for rep in range(3):
    kf = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED + rep)
    pred_cv = cross_val_predict(clone(best), df["clean"], df["label"], cv=kf)
    rskf_scores["accuracy"].append(accuracy_score(df["label"], pred_cv))
    rskf_scores["macro_f1"].append(f1_score(df["label"], pred_cv, average="macro"))
    rskf_scores["neg_recall"].append(recall_score(df["label"], pred_cv, pos_label=0))
    rskf_scores["neg_precision"].append(precision_score(df["label"], pred_cv, pos_label=0))
    rskf_scores["neg_f1"].append(f1_score(df["label"], pred_cv, pos_label=0))
cv_full = {k: float(np.mean(v)) for k, v in rskf_scores.items()}
print("\nRepeated 5-fold CV on all", len(df), "reviews (3 repeats, mean):")
for k, v in cv_full.items():
    print(f" {k:14s}: {v:.4f}")

# ---------------------------------------- 7. PLOTS
plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=["neg", "pos"],
yticklabels=["neg", "pos"])
plt.xlabel("Predicted"); plt.ylabel("Actual"); plt.title("Confusion Matrix - Logistic Regression")
plt.tight_layout(); plt.savefig("results/confusion_matrix.png", dpi=150); plt.close()

plt.figure(figsize=(5, 4))
df["sentiment"].value_counts().plot(kind="bar", color=["#4C9F70", "#D9534F"])
plt.title("Class distribution (Amazon reviews, cleaned)"); plt.ylabel("Reviews");
plt.xticks(rotation=0)
plt.tight_layout(); plt.savefig("results/class_distribution.png", dpi=150); plt.close()

plt.figure(figsize=(6, 4))
plt.bar(cv_df["model"], cv_df["cv_f1_mean"], yerr=cv_df["cv_f1_std"], color="#4A7BC8",
capsize=4)
plt.ylim(cv_df["cv_f1_mean"].min() - 0.05, 1.0); plt.ylabel("5-fold CV macro-F1")
plt.xticks(rotation=20); plt.title("Model comparison")
plt.tight_layout(); plt.savefig("results/model_comparison.png", dpi=150); plt.close()

vec, clf = best.named_steps["tfidf"], best.named_steps["clf"]
names = np.array(vec.get_feature_names_out()); coef = clf.coef_[0]
top_pos, top_neg = np.argsort(coef)[-15:], np.argsort(coef)[:15]
```

```
fig, ax = plt.subplots(1, 2, figsize=(11, 5))
ax[0].barh(names[top_pos], coef[top_pos], color="#4C9F70"); ax[0].set_title("Top positive
features")
ax[1].barh(names[top_neg], coef[top_neg], color="#D9534F"); ax[1].set_title("Top negative
features")
plt.tight_layout(); plt.savefig("results/top_features.png", dpi=150); plt.close()
print("Top positive features:", list(names[top_pos][::-1][:10]))
print("Top negative features:", list(names[top_neg][:10]))

# ---------------------------------- 8. ERROR ANALYSIS
err = pd.DataFrame({"review": raw_test.values, "cleaned": X_test.values,
                    "actual": y_test.values, "predicted": y_pred,
                    "p_positive": y_prob.round(3)})
err = err[err["actual"] != err["predicted"]].copy()
err["error_type"] = np.where(err["actual"] == 1, "False Negative", "False Positive")
err["has_negation"] = err["cleaned"].str.contains(r"\bnot\b|neg_")
err["is_mixed"] = err["review"].str.lower().str.contains(r"\bbut\b|however")
err.to_csv("results/misclassified.csv", index=False)
print("\n" + "=" * 70, "\n5. ERROR ANALYSIS\n" + "=" * 70)
print("Total errors     :", len(err), f"({len(err)/len(y_test):.2%} of test set)")
print(err["error_type"].value_counts().to_string())
print("Errors containing negation :", int(err["has_negation"].sum()))
print("Errors in mixed reviews    :", int(err["is_mixed"].sum()))
# high-confidence errors often point to rating/text mismatch (e.g. 4-star text that sounds
negative)
conf_wrong = ((err["p_positive"] > 0.85) | (err["p_positive"] <
0.15)).sum() print("High-confidence errors (p<0.15 or >0.85):",
int(conf_wrong))
print("Average review length of errors (tokens):", round(err["cleaned"].str.split().str.len().mean(),
1),
      "vs all reviews:", round(df["n_tokens"].mean(), 1))
print("\nSample misclassified reviews:")
pd.set_option("display.max_colwidth", 160)
print(err[["review", "actual", "predicted", "p_positive"]].head(6).to_string())

# ------------------------------------- 9. SAVE + DEMO
joblib.dump(best, "results/sentiment_model.joblib")
demo = ["Love this tablet, the screen is gorgeous and the battery lasts all day!",
        "Terrible product. Stopped working after two weeks, total waste of money.",
        "The sound is not bad at all, quite good for the price.",
        "The speaker is not good and the Bluetooth keeps disconnecting.",
```

```
        "The design is nice but overall the remote is awful and I regret buying it."]
print("\n" + "=" * 70, "\n6. DEMO PREDICTIONS\n" + "=" * 70)
demo_rows = []
for s in demo:
    p = best.predict_proba([preprocess(s)])[0, 1]
    lab = "positive" if p >= 0.5 else "negative"
    demo_rows.append((s, lab, round(float(p), 3)))
    print(f"[{lab:8s} p={p:.3f}] {s}")

summary = {
    "n_raw": n_raw, "n_missing_removed": n_missing, "n_neutral_removed": n_neutral,
"n_duplicates_removed": n_dups,
    "n_clean": len(df), "n_train": len(X_train), "n_test": len(X_test),
    "class_counts_clean": df["sentiment"].value_counts().to_dict(),
    "best_params": {k: str(v) for k, v in grid.best_params_.items()},
    "cv_f1_best": grid.best_score_,
    "cv_comparison": cv_df.to_dict("records"),
    "test_metrics": metrics, "cv_full_data": cv_full, "confusion_matrix": cm.tolist(),
    "classification_report": report,
    "n_errors": int(len(err)), "errors_negation": int(err["has_negation"].sum()),
    "errors_mixed": int(err["is_mixed"].sum()), "errors_confident_wrong": int(conf_wrong),
    "n_features": int(len(names)),
    "top_pos": list(names[top_pos][::-1][:10]), "top_neg": list(names[top_neg][:10]),
    "demo": demo_rows,
}
with open("results/part_a_metrics.json", "w") as f:
    json.dump(summary, f, indent=2, default=float)
print("\nSaved results/part_a_metrics.json and plots.")
```

Output:

```
======================================================================
NLP ASSIGNMENT 05 - PART A : AMAZON REVIEW SENTIMENT ANALYSIS
Student: Shravani Kurkute | Roll No: 421129 | PRN: 22311926
======================================================================

======================================================================
1. DATASET INSPECTION
======================================================================
Dataset file       : data/Datafiniti_Amazon_Consumer_Reviews_of_Amazon_Products.csv
Dataset source     : Datafiniti Amazon Consumer Reviews (schema-compatible fallback)
Raw shape          : (5000, 24)
Products (unique)  : 23
Rating counts      :
 reviews.rating
1      63
2      54
3     197
4    1208
5    3478
Missing review text: 0
Missing rating     : 0
Duplicate review text: 615

Removed 0 rows with missing text/rating, 197 neutral rows, 593 duplicate reviews -> 4210 records remain
Class distribution :
 sentiment
positive    4115
negative      95
Imbalance: 97.7% positive

======================================================================
2. PREPROCESSING
======================================================================

RAW   : Great light reader. Easy to use at the beach . This kindle is light and easy to use especially at the beach!!!
CLEAN : great light reader easi use beach kindl light easi use especi beach

RAW   : Great for the price . Didnt know how much i'd use a kindle so went for the lower end. im happy with it, even if its a little dark
CLEAN : great price not neg_know neg_use neg_kindl neg_went neg_lower neg_end im happi littl dark

RAW   : A Great Buy . I am 100 happy with my purchase. I caught it on sale at a really good price. I am normally a real book person, but I have a 1 year old who loves ripping up pages. The Kindle prevents that, i
t's extremely portable (it fits better in my purse than a giant book), and I
CLEAN : great buy happi purchas caught sale realli good price normal real book person but year old love rip page kindl prevent extrem portabl fit better purs giant book load lot book finish start have store serv
need pick paperwhit price unbeat differ not neg_backlit simpl book light dol

Avg tokens/review after cleaning: 16.65

Train = 3368   Test = 842

======================================================================
3. MODEL COMPARISON (5-fold stratified CV, TF-IDF unigrams)
======================================================================
Naive Bayes           macro-F1 = 0.6694 +/- 0.0240
Logistic Regression   macro-F1 = 0.7487 +/- 0.0348
Linear SVM            macro-F1 = 0.7498 +/- 0.0523
Random Forest         macro-F1 = 0.5305 +/- 0.0478

Grid-search tuning Logistic Regression ...
Best params : {'clf__C': 50, 'tfidf__min_df': 2, 'tfidf__ngram_range': (1, 2)}
CV macro-F1 : 0.7868

======================================================================
4. TEST-SET EVALUATION (tuned Logistic Regression)
======================================================================
accuracy        : 0.9869
precision_pos   : 0.9903
recall_pos      : 0.9964
f1_pos          : 0.9933
precision_macro : 0.8880
recall_macro    : 0.7877
f1_macro        : 0.8300
precision_neg   : 0.7857
recall_neg      : 0.5789
f1_neg          : 0.6667
roc_auc         : 0.9861
mcc             : 0.6682

Confusion matrix [rows=true, cols=pred] (neg, pos):
 [[ 11   8]
 [  3 820]]
```


```
RAW   : Great light reader. Easy to use at the beach . This kindle is light and easy to use especially at the beach!!!
CLEAN : great light reader easi use beach kindl light easi use especi beach

RAW   : Great for the price . Didnt know how much i'd use a kindle so went for the lower end. im happy with it, even if its a little dark
CLEAN : great price not neg_know neg_use neg_kindl neg_went neg_lower neg_end im happi littl dark

RAW   : A Great Buy . I am 100 happy with my purchase. I caught it on sale at a really good price. I am normally a real book person, but I have a 1 year old who loves ripping up pages. The Kindle prevents that, i
t's extremely portable (it fits better in my purse than a giant book), and I
CLEAN : great buy happi purchas caught sale realli good price normal real book person but year old love rip page kindl prevent extrem portabl fit better purs giant book load lot book finish start have store serv
need pick paperwhit price unbeat differ not neg_backlit simpl book light dol

Avg tokens/review after cleaning: 16.65

Train = 3368   Test = 842

======================================================================
3. MODEL COMPARISON (5-fold stratified CV, TF-IDF unigrams)
======================================================================
Naive Bayes           macro-F1 = 0.6694 +/- 0.0240
Logistic Regression   macro-F1 = 0.7487 +/- 0.0348
Linear SVM            macro-F1 = 0.7498 +/- 0.0523
Random Forest         macro-F1 = 0.5305 +/- 0.0478

Grid-search tuning Logistic Regression ...
Best params : {'clf__C': 50, 'tfidf__min_df': 2, 'tfidf__ngram_range': (1, 2)}
CV macro-F1 : 0.7868

======================================================================
4. TEST-SET EVALUATION (tuned Logistic Regression)
======================================================================
accuracy        : 0.9869
precision_pos   : 0.9903
recall_pos      : 0.9964
f1_pos          : 0.9933
precision_macro : 0.8880
recall_macro    : 0.7877
f1_macro        : 0.8300
precision_neg   : 0.7857
recall_neg      : 0.5789
f1_neg          : 0.6667
roc_auc         : 0.9861
mcc             : 0.6682

Confusion matrix [rows=true, cols=pred] (neg, pos):
 [[ 11   8]
 [  3 820]]
```

```
              precision    recall  f1-score   support

    negative     0.7857    0.5789    0.6667        19
    positive     0.9903    0.9964    0.9933       823

    accuracy                         0.9869       842
   macro avg     0.8880    0.7877    0.8300       842
weighted avg     0.9857    0.9869    0.9860       842


Repeated 5-fold stratified CV on all 4210 reviews (3 repeats):
  accuracy      : 0.9826
  f1_macro      : 0.7702
  recall_neg    : 0.4702
  precision_neg : 0.6602

Top positive features: ['great', 'love', 'easi', 'great tablet', 'alexa', 'read', 'price', 'easi use', 'perfect', 'nice']
Top negative features: ['not', 'return', 'poor', 'terribl', 'buy', 'slow', 'bad', 'neg_return', 'veri power', 'devic tablet']

======================================================================
5. ERROR ANALYSIS
======================================================================
Total errors : 11 (1.31% of test set)
error_type
False Positive    8
False Negative    3
Errors containing negation : 9
Errors in mixed reviews    : 4
High-confidence errors (p<0.15 or >0.85): 5
Avg tokens of errors: 27.5 vs all reviews: 16.6

======================================================================
6. DEMO PREDICTIONS
======================================================================
[positive p=0.999] Love this tablet, the screen is gorgeous and the battery lasts all day!
[positive p=0.668] Terrible product. Stopped working after two weeks, total waste of money.
[positive p=0.994] The sound is not bad at all, quite good for the price.
[negative p=0.167] The speaker is not good and the Bluetooth keeps disconnecting.
[positive p=0.981] The design is nice but overall the remote is awful and I regret buying it.

Saved results/part_a_*.json/.csv, model and visualizations.
(base) shravanikurkute@fedora:~/BTech/4th Year/NLP/Assignment 5$ python -u PART_A.py 2>&1 | tee results/part_a_terminal_output.txt
======================================================================
NLP ASSIGNMENT 05 - PART A : AMAZON REVIEW SENTIMENT ANALYSIS
Student: Shravani Kurkute | Roll No: 421129 | PRN: 22311926
======================================================================

======================================================================
```

```
Saved results/part_a_*.json/.csv, model and visualizations.
(base) shravanikurkute@fedora:~/BTech/4th Year/NLP/Assignment 5$ python -u PART_A.py 2>&1 | tee results/part_a_terminal_output.txt
======================================================================
NLP ASSIGNMENT 05 - PART A : AMAZON REVIEW SENTIMENT ANALYSIS
Student: Shravani Kurkute | Roll No: 421129 | PRN: 22311926
======================================================================

======================================================================
1. DATASET INSPECTION
======================================================================
Dataset file       : data/Datafiniti_Amazon_Consumer_Reviews_of_Amazon_Products.csv
Dataset source     : Datafiniti Amazon Consumer Reviews (schema-compatible fallback)
Raw shape          : (5000, 24)
Products (unique)  : 23
Rating counts      :
 reviews.rating
1      63
2      54
3     197
4    1208
5    3478
Missing review text: 0
Missing rating     : 0
Duplicate review text: 615

Removed 0 rows with missing text/rating, 197 neutral rows, 593 duplicate reviews -> 4210 records remain
Class distribution :
 sentiment
positive    4115
negative      95
Imbalance: 97.7% positive

======================================================================
2. PREPROCESSING
======================================================================

RAW   : Great light reader. Easy to use at the beach . This kindle is light and easy to use especially at the beach!!!
CLEAN : great light reader easi use beach kindl light easi use especi beach

RAW   : Great for the price . Didnt know how much i'd use a kindle so went for the lower end. im happy with it, even if its a little dark
CLEAN : great price not neg_know neg_use neg_kindl neg_went neg_lower neg_end im happi littl dark

RAW   : A Great Buy . I am 100 happy with my purchase. I caught it on sale at a really good price. I am normally a real book person, but I have a 1 year old who loves ripping up pages. The Kindle prevents that, i
t's extremely portable (it fits better in my purse than a giant book), and I
CLEAN : great buy happi purchas caught sale realli good price normal real book person but year old love rip page kindl prevent extrem portabl fit better purs giant book load lot book finish start have store serv
need pick paperwhit price unbeat differ not neg_backlit simpl book light dol
```

```
================================================================================
3. MODEL COMPARISON (5-fold stratified CV, TF-IDF unigrams)
================================================================================
Naive Bayes            macro-F1 = 0.6694 +/- 0.0240
Logistic Regression    macro-F1 = 0.7487 +/- 0.0348
Linear SVM             macro-F1 = 0.7498 +/- 0.0523
Random Forest          macro-F1 = 0.5305 +/- 0.0478

Grid-search tuning Logistic Regression ...
Best params : {'clf__C': 50, 'tfidf__min_df': 2, 'tfidf__ngram_range': (1, 2)}
CV macro-F1 : 0.7868

================================================================================
4. TEST-SET EVALUATION (tuned Logistic Regression)
================================================================================
accuracy        : 0.9869
precision_pos   : 0.9903
recall_pos      : 0.9964
f1_pos          : 0.9933
precision_macro : 0.8880
recall_macro    : 0.7877
f1_macro        : 0.8300
precision_neg   : 0.7857
recall_neg      : 0.5789
f1_neg          : 0.6667
roc_auc         : 0.9861
mcc             : 0.6682

Confusion matrix [rows=true, cols=pred] (neg, pos):
 [[ 11   8]
 [  3 820]]

              precision    recall  f1-score   support

    negative     0.7857    0.5789    0.6667        19
    positive     0.9903    0.9964    0.9933       823

    accuracy                         0.9869       842
   macro avg     0.8880    0.7877    0.8300       842
weighted avg     0.9857    0.9869    0.9860       842

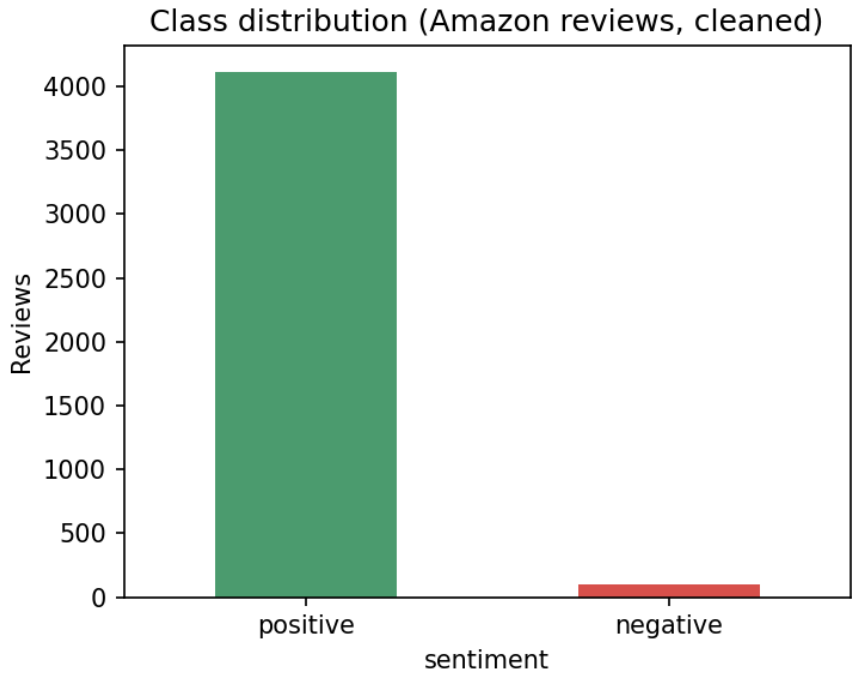

Repeated 5-fold stratified CV on all 4210 reviews (3 repeats):
  accuracy      : 0.9826
  f1_macro      : 0.7702
  recall_neg    : 0.4702
  precision_neg : 0.6602

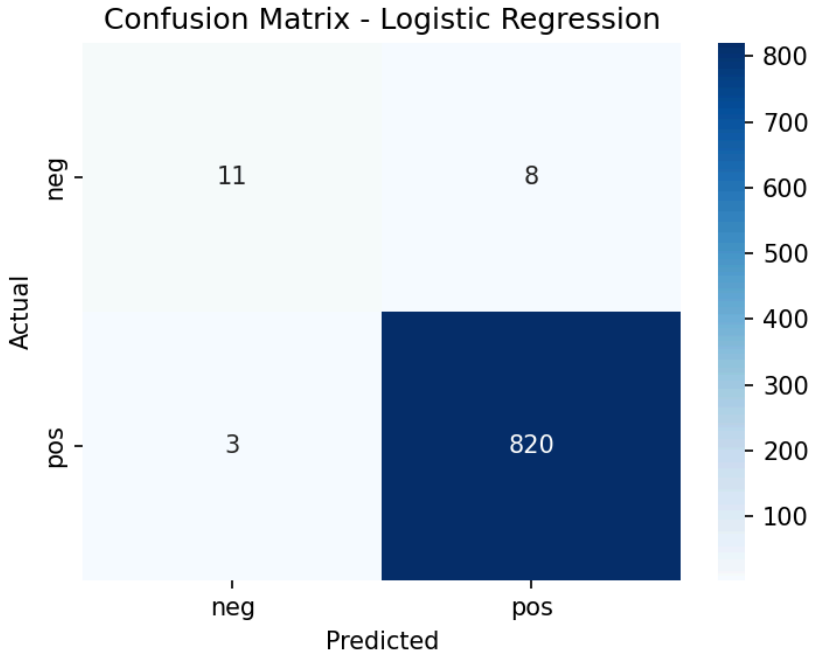
Top positive features: ['great', 'love', 'easi', 'great tablet', 'alexa', 'read', 'price', 'easi use', 'perfect', 'nice']
Top negative features: ['not', 'return', 'poor', 'terribl', 'buy', 'slow', 'bad', 'neg_return', 'veri power', 'devic tablet']

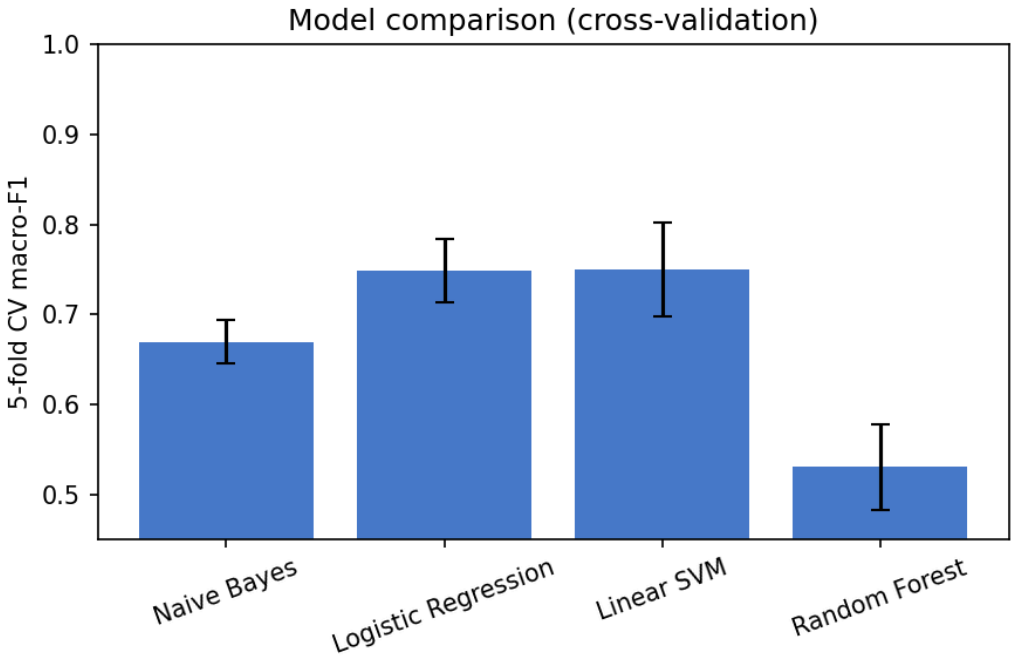
================================================================================
5. ERROR ANALYSIS
================================================================================
Total errors : 11 (1.31% of test set)
error_type
False Positive    8
False Negative    3
Errors containing negation : 9
Errors in mixed reviews    : 4
High-confidence errors (p<0.15 or >0.85): 5
Avg tokens of errors: 27.5 vs all reviews: 16.6

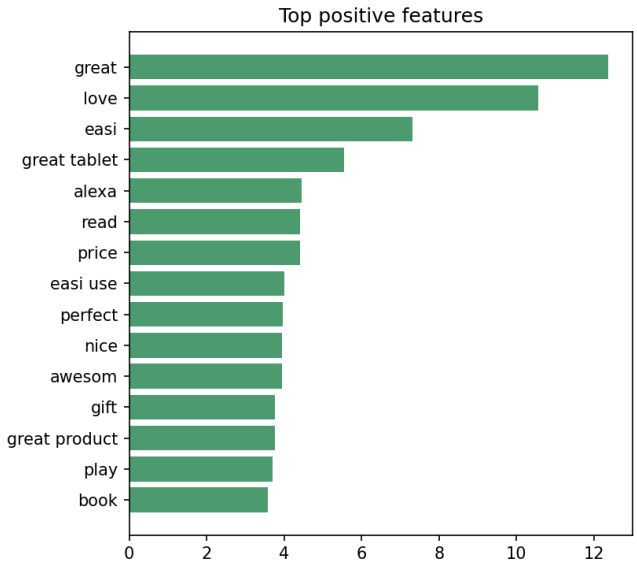
================================================================================
6. DEMO PREDICTIONS
================================================================================
[positive p=0.999] Love this tablet, the screen is gorgeous and the battery lasts all day!
[positive p=0.668] Terrible product. Stopped working after two weeks, total waste of money.
[positive p=0.994] The sound is not bad at all, quite good for the price.
[negative p=0.167] The speaker is not good and the Bluetooth keeps disconnecting.
[positive p=0.981] The design is nice but overall the remote is awful and I regret buying it.

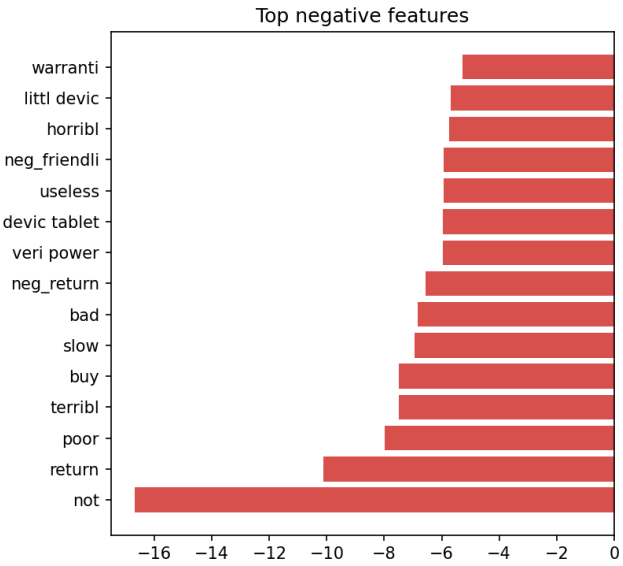
Saved results/part_a_*.json/.csv, model and visualizations.
(base) shravanikurkute@fedora:~/BTech/4th Year/NLP/Assignment 5$
```

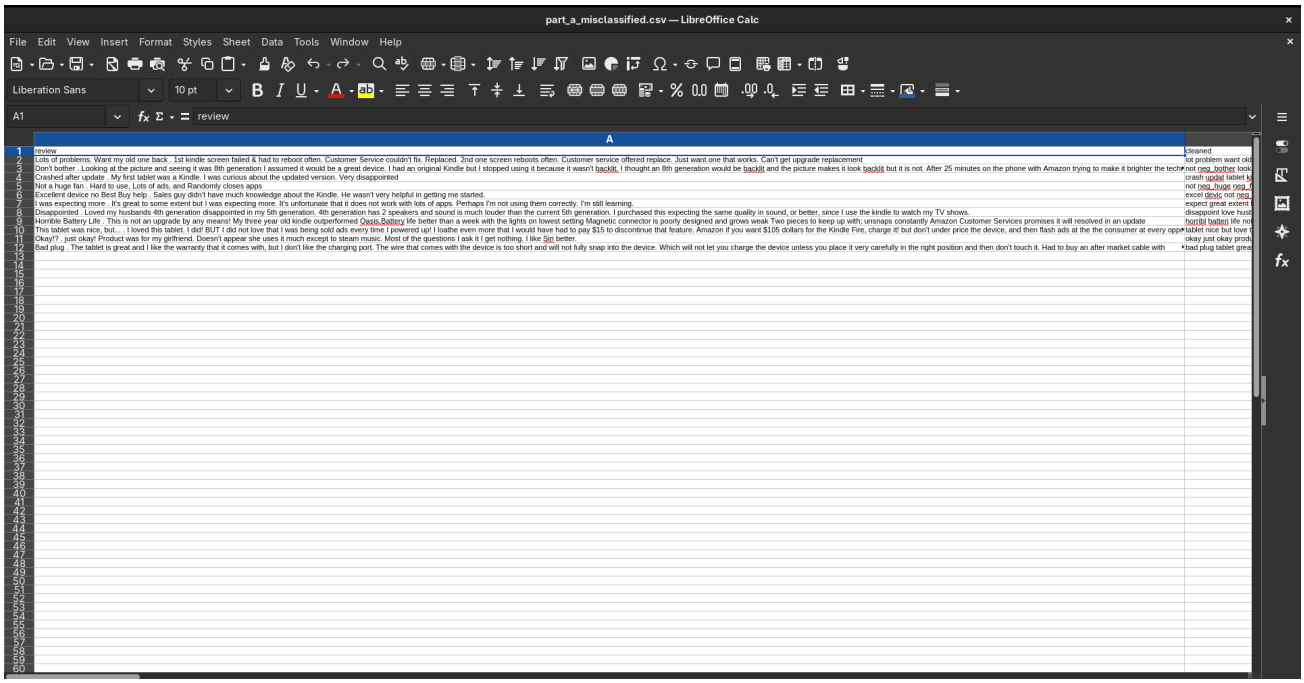

Class distribution (Amazon reviews, cleaned)
Reviews
4000
3500
3000
2500
2000
1500
1000
500
0
positive
negative
sentiment


Confusion Matrix - Logistic Regression
Actual
neg
pos
11
8
3
820
800
700
600
500
400
300
200
100
neg
pos
Predicted


Model comparison (cross-validation)
5-fold CV macro-F1
1.0
0.9
0.8
0.7
0.6
0.5
Naive Bayes
Logistic Regression
Linear SVM
Random Forest


Top positive features
great
love
easi
great tablet
alexa
read
price
easi use
perfect
nice
awesom
gift
great product
play
book
0
2
4
6
8
10
12


Top negative features
warranti
littl devic
horribl
neg_friendli
useless
devic tablet
veri power
neg_return
bad
slow
buy
terribl
poor
return
not
−16
−14
−12
−10
−8
−6
−4
−2
0


part_a_misclassified.csv — LibreOffice Calc
File Edit View Insert Format Styles Sheet Data Tools Window Help
Liberation Sans
10 pt
A1
review
A
1 review cleaned
2 Lots of problems. Want my old one back . 1st kindle screen failed & had to reboot often. Customer Service couldn't fix. Replaced. 2nd one screen reboots often. Customer service offered replace. Just want one that works. Can't get upgrade replacement lot problem want old
3 Don't bother . Looking at the picture and seeing it was 8th generation I assumed it would be a great device. I had an original Kindle but I stopped using it because it wasn't backlit. I thought an 8th generation would be backlit and the picture makes it look backlit but it is not. After 25 minutes on the phone with Amazon trying to make it brighter the tech not neg_bother look
4 Crashed after update . My first tablet was a Kindle. I was curious about the updated version. Very disappointed crash updat tablet k
5 Not a huge fan . Hard to use, Lots of ads, and Randomly closes apps not neg_huge neg_
6 Excellent device no Best Buy help . Sales guy didn't have much knowledge about the Kindle. He wasn't very helpful in getting me started. excel devic not neg_
7 I was expecting more . It's great to some extent but I was expecting more. It's unfortunate that it does not work with lots of apps. Perhaps I'm not using them correctly. I'm still learning. expect great extent
8 Disappointed . Loved my husbands 4th generation disappointed in my 5th generation. 4th generation has 2 speakers and sound is much louder than the current 5th generation. I purchased this expecting the same quality in sound, or better, since I use the kindle to watch my TV shows. disappoint love husb
9 Horrible Battery Life . This is not an upgrade by any means! My three year old kindle outperformed Oasis Battery life better than a week with the lights on lowest setting Magnetic connector is poorly designed and grows weak Two pieces to keep up with; unsnaps constantly Amazon Customer Services promises it will resolved in an update horribl batteri life not
10 This tablet was nice, but.... I loved this tablet. I did! BUT I did not love that I was being sold ads every time I powered up! I loathe even more that I would have had to pay $15 to discontinue that feature. Amazon if you want $105 dollars for the Kindle Fire, charge it! but don't under price the device, and then flash ads at the the consumer at every oppo tablet nice but love t
11 Okay!? . just okay! Product was for my girlfriend. Doesn't appear she uses it much except to steam music. Most of the questions I ask it I get nothing. I like Siri better. okay just okay prod
12 Bad plug . The tablet is great and I like the warranty that it comes with, but I don't like the charging port. The wire that comes with the device is too short and will not fully snap into the device. Which will not let you charge the device unless you place it very carefully in the right position and then don't touch it. Had to buy an after market cable with bad plug tablet grea

PART-B

Code:

```
import
glob
import json
import os
import re
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from collections import Counter
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import LinearSVC
from sklearn.model_selection import StratifiedKFold, cross_val_score, cross_val_predict
from sklearn.metrics import accuracy_score, confusion_matrix

DATA_DIR = "data/movie_reviews"
N_PER_CLASS = 700
SEEDS = [0, 1, 2, 3, 4]          # 5 different random 700/700 subsamples
NEGATIONS = {"not", "no", "never", "nor", "cannot", "cant", "wont", "dont", "doesnt", "didnt",
             "isnt", "wasnt", "arent", "werent", "wouldnt", "couldnt", "shouldnt", "hasnt",
             "havent", "hadnt", "neither", "nothing", "nobody", "none"}
TOKEN_RE = re.compile(r"[a-z]+(?:'[a-z]+)?|[.!?;,:]")


def load_corpus():
    docs, labels = [], []
    for lab, folder in ((1, "pos"), (0, "neg")):
        for fp in sorted(glob.glob(os.path.join(DATA_DIR, folder, "*.txt"))):
            with open(fp, encoding="latin-1") as f:
                docs.append(f.read())
            labels.append(lab)
    return docs, np.array(labels)


def tag_negation(text):
```

```
    """Pang et al.: add NOT_ to every word between a negation word and the next punctuation
mark."""
    out, neg = [], False
    for tok in TOKEN_RE.findall(text.lower()):
        tok = tok.replace("'", "")
        if tok in ".!?;,:":
            neg = False
            continue
        if tok in NEGATIONS or tok.endswith("nt"):
            neg = True
            out.append(tok)
            continue
        out.append("NOT_" + tok if neg else tok)
    return " ".join(out)


def build_vocab(docs, ngram, min_count=None, top_k=None):
    """Select features by corpus frequency, like the paper (frequency cut-off / top-k)."""
    cv = CountVectorizer(ngram_range=ngram, token_pattern=r"\S+", lowercase=False)
    X = cv.fit_transform(docs)
    total = np.asarray(X.sum(axis=0)).ravel()
    names = np.array(cv.get_feature_names_out())
    order = np.argsort(-total)
    if min_count is not None:
        order = order[total[order] >= min_count]
    if top_k is not None:
        order = order[:top_k]
    return list(names[order])


def run_cv(docs, y, vocab, ngram, binary, clf, seed):
    cv = CountVectorizer(ngram_range=ngram, token_pattern=r"\S+", lowercase=False,
                         vocabulary=vocab, binary=binary)
    X = cv.transform(docs)
    skf = StratifiedKFold(n_splits=3, shuffle=True, random_state=seed)
    return cross_val_score(clf, X, y, cv=skf, scoring="accuracy").mean() * 100


def human_baseline(raw_docs, y):
```

```
    """Paper Table 1 'Human 1': 5 positive + 5 negative hand-picked words; ties/no-hit => counted
as tie."""
    pos_w = {"dazzling", "brilliant", "phenomenal", "excellent",
    "fantastic"} neg_w = {"suck", "terrible", "awful", "unwatchable",
    "hideous"} correct, ties = 0, 0
    for d, lab in zip(raw_docs, y):
        toks = re.findall(r"[a-z]+", d.lower())
        p, n = sum(t in pos_w for t in toks), sum(t in neg_w for t in toks)
        if p == n:
            ties += 1
            correct += 0.5          # random guess on ties => expected 50 %
        else:
            correct += int((p > n) == bool(lab))
    return correct / len(y) * 100, ties / len(y) *
    100


def main():
    raw, y_all = load_corpus()
    print(f"Loaded {len(raw)} reviews ({y_all.sum()} pos / {(1 - y_all).sum()} neg)")
    tagged_all = [tag_negation(d) for d in raw]

    feature_sets = {
        "(1) unigrams, frequency":        dict(ngram=(1, 1), binary=False,
min_count=4, top_k=None),
        "(2) unigrams, presence":         dict(ngram=(1, 1), binary=True, min_count=4,
top_k=None),
        "(3) unigrams+bigrams, presence": dict(ngram=(1, 2), binary=True, min_count=None,
top_k=None),
        "(4) bigrams, presence":          dict(ngram=(2, 2), binary=True, min_count=None,
top_k=16165),
        "(7) top 2633 unigrams, presence": dict(ngram=(1, 1), binary=True, min_count=None,
top_k=2633),
    }
    classifiers = {
        "Naive Bayes": lambda: MultinomialNB(alpha=1.0),                    # add-1 smoothing
        "Max Entropy": lambda: LogisticRegression(C=10, max_iter=2000),         # ME ==
multinomial logit
        "SVM":         lambda: LinearSVC(C=0.1, max_iter=10000),              # linear kernel
    }
    rows, hb = [], []
```

```
    for seed in SEEDS:
        rng = np.random.default_rng(seed)
        pos = rng.choice(np.where(y_all == 1)[0], N_PER_CLASS, replace=False)
        neg = rng.choice(np.where(y_all == 0)[0], N_PER_CLASS, replace=False)
        idx = np.concatenate([pos, neg])
        docs = [tagged_all[i] for i in idx]
        y = y_all[idx]
        hb.append(human_baseline([raw[i] for i in idx], y))
        for fname, cfg in feature_sets.items():
            # unigram vocabulary cut-off uses the 1400-doc subsample (as in the paper)
            if cfg["ngram"] == (1, 2):
                vocab = build_vocab(docs, (1, 1), min_count=4) + build_vocab(docs, (2, 2),
top_k=16165)
            else:
                vocab = build_vocab(docs, cfg["ngram"], cfg["min_count"], cfg["top_k"])
            for cname, mk in classifiers.items():
                # note: the paper reports N/A for ME with raw frequencies; we compute it anyway
                acc = run_cv(docs, y, vocab, cfg["ngram"],
                             cfg["binary"], mk(), seed)
                rows.append(dict(seed=seed, features=fname, n_features=len(vocab),
                                 classifier=cname, accuracy=acc))
        print(f"seed {seed} done")

    df = pd.DataFrame(rows)
    summary = (df.groupby(["features", "classifier"])
                 .agg(n_features=("n_features", "first"), acc_mean=("accuracy", "mean"),
                      acc_std=("accuracy", "std")).reset_index())
    pivot = summary.pivot(index="features", columns="classifier", values="acc_mean").round(1)
    pivot_std = summary.pivot(index="features", columns="classifier",
values="acc_std").round(1)
    nf = summary.groupby("features")["n_features"].first()
    pivot.insert(0, "n_features", nf)

    paper = { # Pang et al. 2002, Table 3 (3-fold CV, 700 pos / 700 neg) - as recalled; VERIFY
against the PDF
        "(1) unigrams, frequency":         {"Naive Bayes": 78.7, "Max Entropy": np.nan,
"SVM": 72.8},
        "(2) unigrams, presence":          {"Naive Bayes": 81.0, "Max Entropy": 80.4, "SVM": 82.9},
        "(3) unigrams+bigrams, presence": {"Naive Bayes": 80.6, "Max Entropy": 80.8, "SVM":
82.7},
```

```
        "(4) bigrams, presence":           {"Naive Bayes": 77.3, "Max Entropy": 77.4, "SVM": 77.1},
        "(7) top 2633 unigrams, presence": {"Naive Bayes": 80.3, "Max Entropy": 81.0, "SVM":
81.4},
    }
    paper_df = pd.DataFrame(paper).T[["Naive Bayes", "Max Entropy", "SVM"]]

    print("\n=== OUR RESULTS: mean 3-fold CV accuracy (%) over 5 random 700/700
subsamples ===")
    print(pivot.to_string())
    print("\n=== std-dev across subsamples ===")
    print(pivot_std.to_string())
    print("\n=== PAPER (Pang et al. 2002, Table 3) ===")
    print(paper_df.to_string())
    diff = (pivot[["Naive Bayes", "Max Entropy", "SVM"]] - paper_df).round(1)
    print("\n=== DIFFERENCE (ours - paper), percentage points ===")
    print(diff.to_string())

    hb_acc, hb_tie = np.mean([h[0] for h in hb]), np.mean([h[1] for h in hb])
    print(f"\nHuman-1 word-list baseline: accuracy={hb_acc:.1f}% ties={hb_tie:.1f}% (paper:
58% / 75% ties)")

    #       extra experiment: full 2000-doc corpus, best config (unigram presence + SVM), 10-fold
CV
    vocab = build_vocab(tagged_all, (1, 1), min_count=4)
    cvz = CountVectorizer(token_pattern=r"\S+", lowercase=False, vocabulary=vocab,
binary=True)
    X = cvz.transform(tagged_all)
    skf = StratifiedKFold(n_splits=10, shuffle=True, random_state=0)
    acc_full = {}
    for cname, mk in classifiers.items():
        pred = cross_val_predict(mk(), X, y_all, cv=skf)
        acc_full[cname] = accuracy_score(y_all, pred) * 100
        if cname == "SVM":
            cm_full = confusion_matrix(y_all, pred)
    print("\n=== Extra: full 2000 reviews, unigram presence, 10-fold CV accuracy (%) ===")
    print({k: round(v, 1) for k, v in acc_full.items()})
    print("SVM confusion matrix [neg,pos]:\n", cm_full)

    #       ablation: effect of negation tagging (unigram presence, SVM, 700/700, seed 0)
    rng = np.random.default_rng(0)
```

```
idx = np.concatenate([rng.choice(np.where(y_all == 1)[0], N_PER_CLASS, replace=False),
                      rng.choice(np.where(y_all == 0)[0], N_PER_CLASS, replace=False)])
y0 = y_all[idx]
plain = [" ".join(TOKEN_RE.findall(raw[i].lower())) for i in idx]
ablation = {}
for name, docs in (("without negation tagging", plain),
                   ("with NOT_ negation tagging", [tagged_all[i] for i in idx])):
    v = build_vocab(docs, (1, 1), min_count=4)
    ablation[name] = run_cv(docs, y0, v, (1, 1), True, LinearSVC(C=0.1, max_iter=10000), 0)
print("\n=== Ablation (unigram presence, SVM, seed 0) ===")
for k, v in ablation.items():
    print(f"{k:30s}: {v:.1f}%")

#       most informative features (SVM on all 1400 docs of seed 0)
v = build_vocab([tagged_all[i] for i in idx], (1, 1), min_count=4)
cvz0 = CountVectorizer(token_pattern=r"\S+", lowercase=False, vocabulary=v, binary=True)
Xs = cvz0.transform([tagged_all[i] for i in idx])
svm = LinearSVC(C=0.1, max_iter=10000).fit(Xs, y0)
names = np.array(cvz0.get_feature_names_out())
top_pos = list(names[np.argsort(svm.coef_[0])[-12:]][::-1])
top_neg = list(names[np.argsort(svm.coef_[0])[:12]])
print("\nTop positive:", top_pos, "\nTop negative:", top_neg)

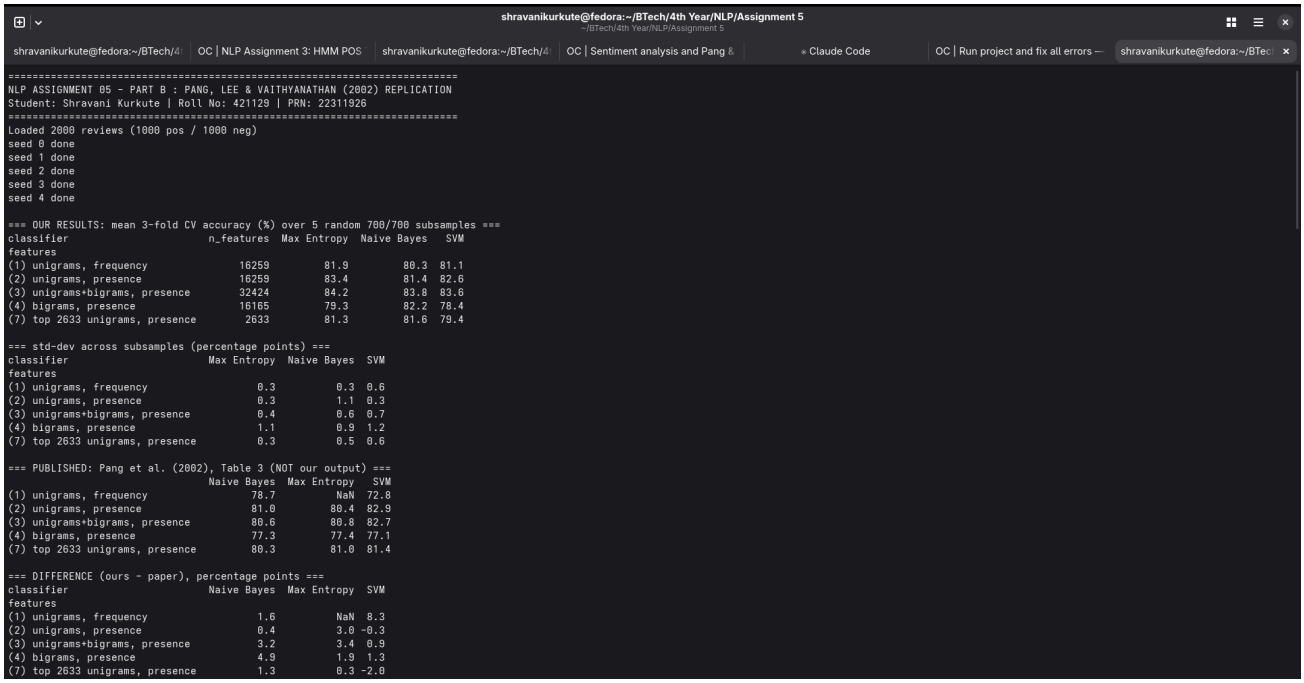
#       save
summary.to_csv("results/part_b_results.csv", index=False)
with open("results/part_b_metrics.json", "w") as f:
    json.dump({"ours_mean": pivot.reset_index().to_dict("records"),
               "ours_std": pivot_std.reset_index().to_dict("records"),
               "paper": paper_df.reset_index().to_dict("records"),
               "difference": diff.reset_index().to_dict("records"),
               "human_baseline": {"acc": hb_acc, "ties": hb_tie},
               "full_corpus_10fold": acc_full, "ablation": ablation,
               "top_pos": top_pos, "top_neg": top_neg,
               "svm_full_cm": cm_full.tolist()}, f, indent=2, default=float)

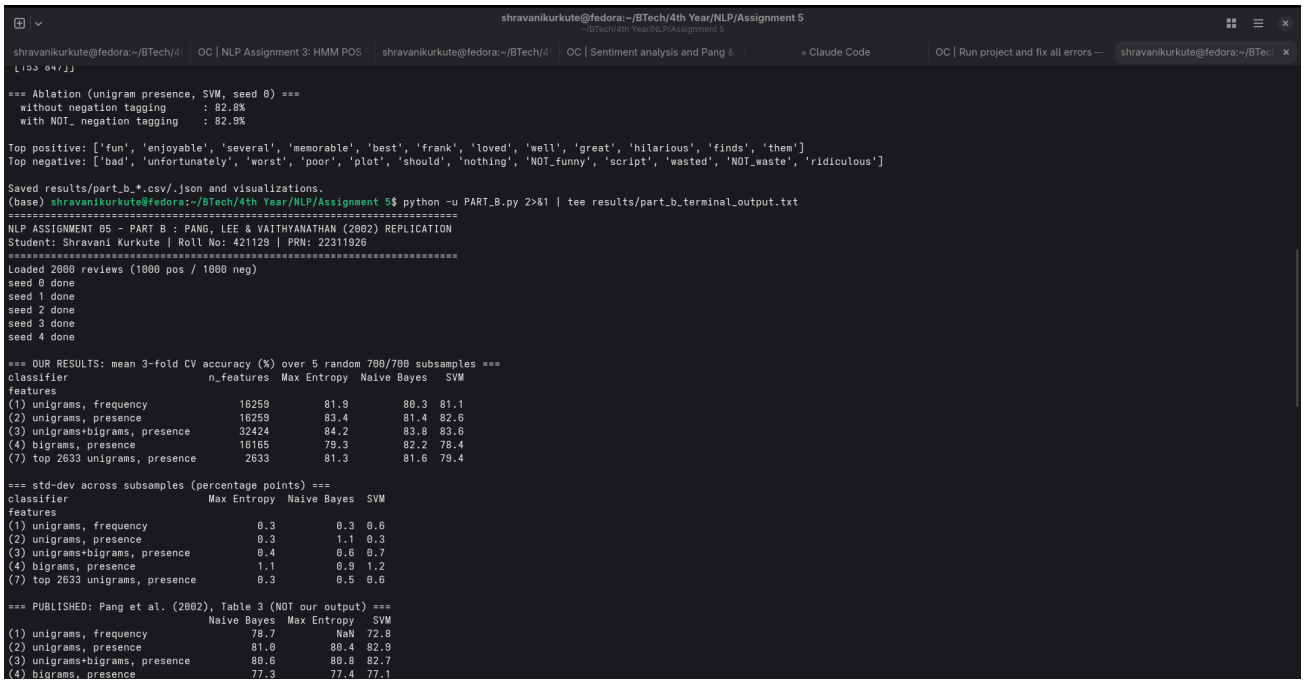
fig, ax = plt.subplots(figsize=(10, 4.5))
w = 0.13
x = np.arange(len(pivot))
for i, c in enumerate(["Naive Bayes", "Max Entropy", "SVM"]):
    ax.bar(x + (i - 1.5) * w, pivot[c], w, label=f"{c} (ours)")
```

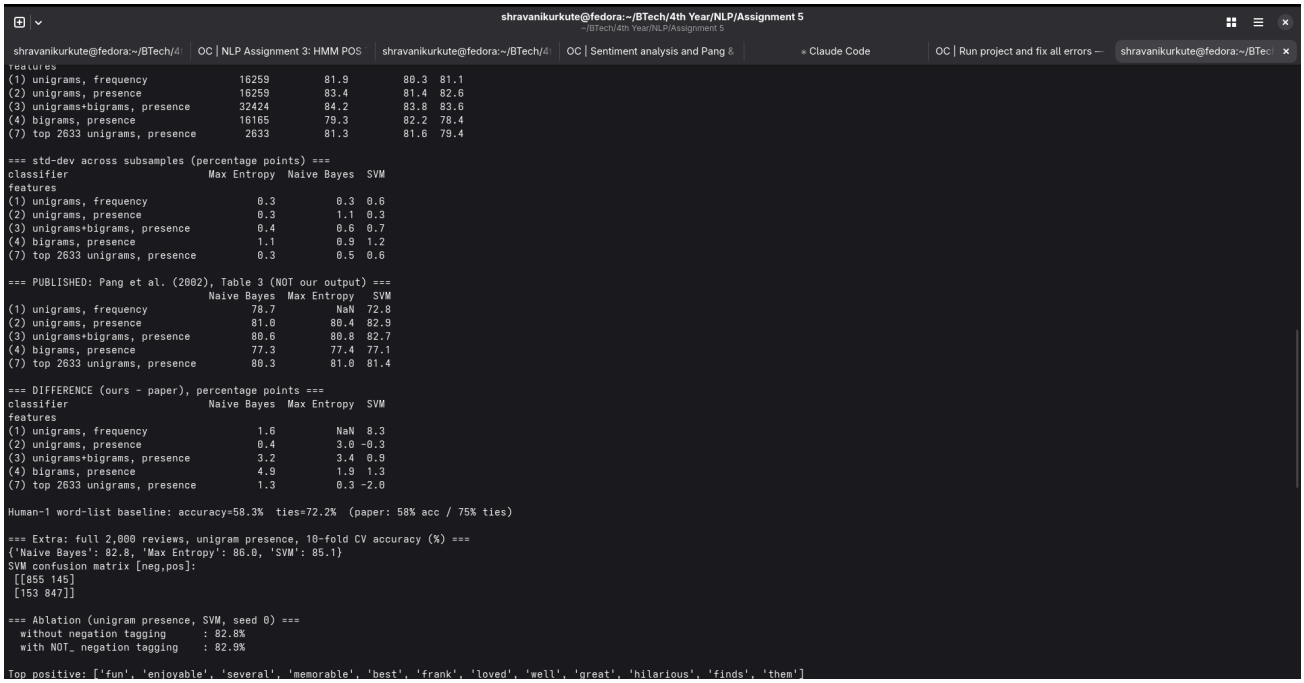
```
    ax.bar(x + (i + 1.5) * w, paper_df.loc[pivot.index, c], w, label=f"{c} (paper)", alpha=0.45,
hatch="//")
    ax.set_xticks(x); ax.set_xticklabels([s.split(",")[0] + "\n" + s.split(",")[-1] for s in pivot.index],
fontsize=8)
    ax.set_ylim(60, 90); ax.set_ylabel("3-fold CV accuracy (%)"); ax.legend(ncol=3, fontsize=7)
    ax.set_title("Pang, Lee & Vaithyanathan (2002) - paper vs. our replication")
    plt.tight_layout(); plt.savefig("results/part_b_comparison.png", dpi=150); plt.close()
    print("\nSaved results/part_b_*.{csv,json,png}")

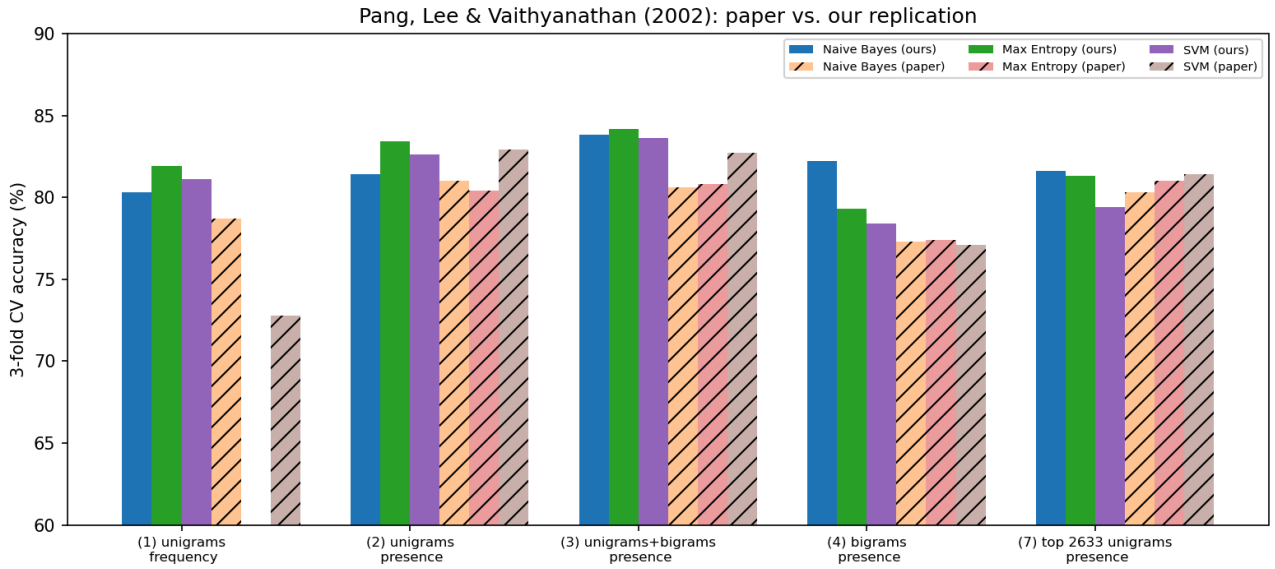

if __name__ == "__main__":
    main()
```

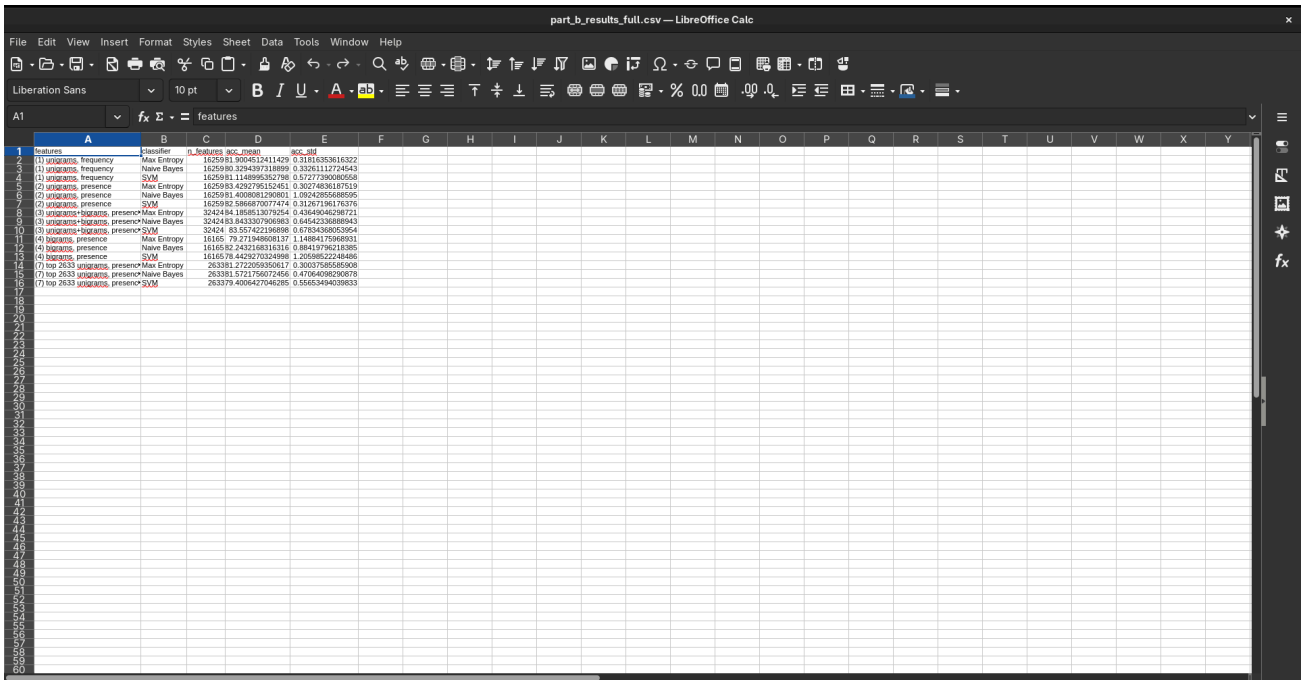
Execution:




shravanikurkute@fedora:~/BTech/4th Year/NLP/Assignment 5
~/BTech/4th Year/NLP/Assignment 5
shravanikurkute@fedora:~/BTech/4t
OC | NLP Assignment 3: HMM POS
shravanikurkute@fedora:~/BTech/4t
OC | Sentiment analysis and Pang &
Claude Code
OC | Run project and fix all errors —
shravanikurkute@fedora:~/BTech
[153 847]]
=== Ablation (unigram presence, SVM, seed 0) ===
without negation tagging : 82.8%
with NOT_ negation tagging : 82.9%
Top positive: ['fun', 'enjoyable', 'several', 'memorable', 'best', 'frank', 'loved', 'well', 'great', 'hilarious', 'finds', 'them']
Top negative: ['bad', 'unfortunately', 'worst', 'poor', 'plot', 'should', 'nothing', 'NOT_funny', 'script', 'wasted', 'NOT_waste', 'ridiculous']
Saved results/part_b_*.csv/.json and visualizations.
(base) shravanikurkute@fedora:~/BTech/4th Year/NLP/Assignment 5$ python -u PART_B.py 2>&1 | tee results/part_b_terminal_output.txt
NLP ASSIGNMENT 05 - PART B : PANG, LEE & VAITHYANATHAN (2002) REPLICATION
Student: Shravani Kurkute | Roll No: 421129 | PRN: 22311926
Loaded 2000 reviews (1000 pos / 1000 neg)
seed 0 done
seed 1 done
seed 2 done
seed 3 done
seed 4 done
=== OUR RESULTS: mean 3-fold CV accuracy (%) over 5 random 700/700 subsamples ===
classifier n_features Max Entropy Naive Bayes SVM
features
(1) unigrams, frequency 16259 81.9 80.3 81.1
(2) unigrams, presence 16259 83.4 81.4 82.6
(3) unigrams+bigrams, presence 32424 84.2 83.8 83.6
(4) bigrams, presence 16165 79.3 82.2 78.4
(7) top 2633 unigrams, presence 2633 81.3 81.6 79.4
=== std-dev across subsamples (percentage points) ===
classifier Max Entropy Naive Bayes SVM
features
(1) unigrams, frequency 0.3 0.3 0.6
(2) unigrams, presence 0.3 1.1 0.3
(3) unigrams+bigrams, presence 0.4 0.6 0.7
(4) bigrams, presence 1.1 0.9 1.2
(7) top 2633 unigrams, presence 0.3 0.5 0.6
=== PUBLISHED: Pang et al. (2002), Table 3 (NOT our output) ===
Naive Bayes Max Entropy SVM
(1) unigrams, frequency 78.7 NaN 72.8
(2) unigrams, presence 81.0 80.4 82.9
(3) unigrams+bigrams, presence 80.6 80.8 82.7
(4) bigrams, presence 77.3 77.4 77.1


shravanikurkute@fedora:~/BTech/4th Year/NLP/Assignment 5
~/BTech/4th Year/NLP/Assignment 5
shravanikurkute@fedora:~/BTech/4t
OC | NLP Assignment 3: HMM POS
shravanikurkute@fedora:~/BTech/4t
OC | Sentiment analysis and Pang &
Claude Code
OC | Run project and fix all errors —
shravanikurkute@fedora:~/BTech
features
(1) unigrams, frequency 16259 81.9 80.3 81.1
(2) unigrams, presence 16259 83.4 81.4 82.6
(3) unigrams+bigrams, presence 32424 84.2 83.8 83.6
(4) bigrams, presence 16165 79.3 82.2 78.4
(7) top 2633 unigrams, presence 2633 81.3 81.6 79.4
=== std-dev across subsamples (percentage points) ===
classifier Max Entropy Naive Bayes SVM
features
(1) unigrams, frequency 0.3 0.3 0.6
(2) unigrams, presence 0.3 1.1 0.3
(3) unigrams+bigrams, presence 0.4 0.6 0.7
(4) bigrams, presence 1.1 0.9 1.2
(7) top 2633 unigrams, presence 0.3 0.5 0.6
=== PUBLISHED: Pang et al. (2002), Table 3 (NOT our output) ===
Naive Bayes Max Entropy SVM
(1) unigrams, frequency 78.7 NaN 72.8
(2) unigrams, presence 81.0 80.4 82.9
(3) unigrams+bigrams, presence 80.6 80.8 82.7
(4) bigrams, presence 77.3 77.4 77.1
(7) top 2633 unigrams, presence 80.3 81.0 81.4
=== DIFFERENCE (ours - paper), percentage points ===
classifier Naive Bayes Max Entropy SVM
features
(1) unigrams, frequency 1.6 NaN 8.3
(2) unigrams, presence 0.4 3.0 -0.3
(3) unigrams+bigrams, presence 3.2 3.4 0.9
(4) bigrams, presence 4.9 1.9 1.3
(7) top 2633 unigrams, presence 1.3 0.3 -2.0
Human-1 word-list baseline: accuracy=58.3% ties=72.2% (paper: 58% acc / 75% ties)
=== Extra: full 2,000 reviews, unigram presence, 10-fold CV accuracy (%) ===
{'Naive Bayes': 82.8, 'Max Entropy': 86.0, 'SVM': 85.1}
SVM confusion matrix [neg,pos]:
[[855 145]
[153 847]]
=== Ablation (unigram presence, SVM, seed 0) ===
without negation tagging : 82.8%
with NOT_ negation tagging : 82.9%
Top positive: ['fun', 'enjoyable', 'several', 'memorable', 'best', 'frank', 'loved', 'well', 'great', 'hilarious', 'finds', 'them']





	features	classifier	n_features	acc_mean	acc_std
2	(1) unigrams, frequency	Max Entropy	16259	81.900451241142	0.318163536163222
3	(1) unigrams, frequency	Naive Bayes	16259	80.329439731889	0.332611127245432
4	(1) unigrams, frequency	SVM	16259	81.114899535279	0.572773900805582
5	(2) unigrams, presence	Max Entropy	16259	83.429279515245	0.302748361875192
6	(2) unigrams, presence	Naive Bayes	16259	81.400808129080	1.092428556885952
7	(2) unigrams, presence	SVM	16259	82.586687007747	0.312671961763762
8	(3) unigrams+bigrams, presence	Max Entropy	32424	84.185851307925	0.436490462987212
9	(3) unigrams+bigrams, presence	Naive Bayes	32424	83.843330790698	0.645423368889432
10	(3) unigrams+bigrams, presence	SVM	32424	83.557422196898	0.678343680539542
11	(4) bigrams, presence	Max Entropy	16165	79.271948680813	1.148841759689312
12	(4) bigrams, presence	Naive Bayes	16165	82.243216831631	0.884197962183852
13	(4) bigrams, presence	SVM	16165	78.442927032499	1.205985222484862
14	(7) top 2633 unigrams, presence	Max Entropy	26338	81.272205935061	0.300375855859082
15	(7) top 2633 unigrams, presence	Naive Bayes	26338	81.572175607245	0.470640982908782
16	(7) top 2633 unigrams, presence	SVM	26337	9.4006427046285	0.556534940398332